

Day 1 — Accessible Front-End Foundations

A practical, lab-based introduction to implementing accessibility from the ground up. Today we connect your existing screen reader expertise directly to frontend code decisions.

Core Principles

- Semantic HTML before ARIA
- Test experience first, inspect code second
- Fix implementation, then retest

Today's Files

- 01-bad-page.html
- 02-good-page.html
- 03-structure-exercise-starter.html
- 06-mini-challenge-starter.html

TODAY'S APPROACH

Today's Learning Approach

This session is structured around audit-and-fix labs rather than passive instruction. Screen reader feedback is not just a verification tool — it is the primary guide for understanding what the code is or is not communicating.

01

Guided Audit Labs

Each exercise begins with a real page. You will audit it with NVDA or JAWS before touching any code, building a user-experience picture first.

02

Screen Reader Feedback Guides Review

What the screen reader announces — or fails to announce — becomes the lens through which we examine HTML structure and ARIA usage.

03

UX to Frontend Code

Every example explicitly connects the user's experience to a specific frontend decision: element choice, attribute, or structure.

CONTEXT

Why This Session Is Different

Most accessibility training begins with awareness. This session begins where that ends — you already understand disability, assistive technology, and WCAG. Here, the focus is implementation precision.



Bridging Theory to Practice

This session moves beyond conceptual understanding into hands-on implementation: writing real code, testing with real assistive technologies, and fixing real failures.



Frontend Implementation Focus

Every concept is framed as a frontend decision: which HTML element to choose, when to add ARIA, how to structure a page for assistive technology.



Screen Readers as Learning Tools

NVDA and JAWS are not just testing instruments today — they are the primary feedback mechanism for understanding whether code is working as intended.

Day 1 Outcomes

By the end of today's session, you will be able to make deliberate, confident frontend decisions that produce accessible, testable results with real assistive technologies.

1

WCAG 2.2 in Frontend Language

Translate the four POUR principles into specific, actionable frontend decisions rather than abstract compliance checklists.

2

Semantic Page Structure

Build page architecture using landmarks, headings, and native HTML elements that assistive technologies can navigate reliably.

3

Native HTML First

Write meaningful links, correct alt text, and properly labelled controls using native HTML before reaching for ARIA.

4

ARIA-Lite Practice

Apply ARIA deliberately — only to enhance correct HTML, communicate state, and fill gaps that native elements cannot address.

WCAG 2.2

WCAG in Frontend Language

WCAG 2.2 is built on four principles. Reframed as frontend questions, they become precise, testable checks rather than policy statements.

Perceivable

Can the user perceive the content? Is information available through more than one sensory channel?



Operable

Can the user operate the interface? Does every control work with keyboard and assistive technology?

Understandable

Can the user understand the content and interaction? Are labels, errors, and instructions unambiguous?



Robust

Can assistive technology reliably interpret the code? Does HTML expose correct name, role, and state?

Perceivable

Perceivability means content is available to users regardless of the sensory channel they rely on. Failures here are often invisible to sighted, mouse-using developers.

What It Means

Information must exist in a form that assistive technology can access and convey. Visual presentation alone is never sufficient.

Frontend Implications

- Images need meaningful alt text — not filenames or empty strings on informative images
- Information conveyed by colour alone fails users who cannot perceive colour differences
- Layout and spatial position cannot be the sole carrier of meaning
- Text alternatives must describe function or meaning, not just appearance

Operable

Operability means every interactive control is reachable and usable without a mouse. Keyboard access is the baseline — and the most common place frontend code breaks accessibility.

All Controls Must Work with Keyboard

If a control cannot be reached or activated with `Tab`, `Enter`, and `Space`, it is inaccessible regardless of its visual appearance.

Focus Order Should Be Logical

The tab sequence must follow the visual and logical reading order of the page. Unexpected focus jumps create disorientation for keyboard and screen reader users.

Interactive Elements Must Be Reachable

Controls hidden visually but present in the DOM, or visible but removed from tab order, both fail operability. Visibility and focusability must align intentionally.

Understandable

Understandability means users can make sense of both the content and the interface behaviour. This principle is most commonly violated by vague labels, unclear errors, and unpredictable interactions.

Links, Labels, and Instructions Must Be Clear

"Click here", "Read more", and "Submit" without context fail screen reader users who navigate by element type. Every label must communicate purpose independently.

Page Structure Should Feel Predictable

Consistent navigation, heading hierarchy, and landmark placement allow users to build a mental model of the page and move through it with confidence.

Error and Action Text Must Explain What to Do

"Error" or a red border alone is not sufficient. Error messages must identify the field, describe the problem, and suggest a correction.

- 📝 Speaker notes: Ask the participant to navigate 01-bad-page.html using NVDA's Elements List (Insert+F7 in NVDA) and list all links. Ask: "Which of those link texts makes sense out of context?" This directly connects the Understandable principle to the link text patterns they will fix later. Mention that error handling will appear in a later session — today's focus is labels and structure. Ask: "If you heard 'click here' three times in a row, how would you know which to activate?"

Robust

Robustness is the principle most directly tied to frontend code quality. Assistive technologies depend on reliable, standards-conformant HTML to derive meaning. Clever visual hacks do not transfer to the accessibility tree.

HTML Must Expose Correct Meaning

The element you choose determines what role the browser exposes in the accessibility tree. A `<div>` has no implicit role; a `<button>` has role button.

Controls Need Correct Name, Role, and State

Every interactive element must have an accessible name, a meaningful role, and — where applicable — communicated state such as `aria-expanded` or `aria-checked`.

Assistive Technology Depends on Reliable Structure

NVDA and JAWS construct their virtual buffers from the accessibility tree. Malformed or inconsistent HTML produces unreliable announcements regardless of visual correctness.

EXERCISE

Exercise: Map Problems to WCAG

Before writing a single line of code, practise translating what you observe through a screen reader into precise WCAG language and frontend decisions. This is the audit-first discipline.

Your Task

1. Open `01-bad-page.html` in your browser
2. Audit the page using NVDA or JAWS — do not inspect the source yet
3. List at least four accessibility problems you encounter
4. Map each problem to its POUR principle
5. Describe the user experience impact in one sentence
6. Identify the likely frontend cause for each issue

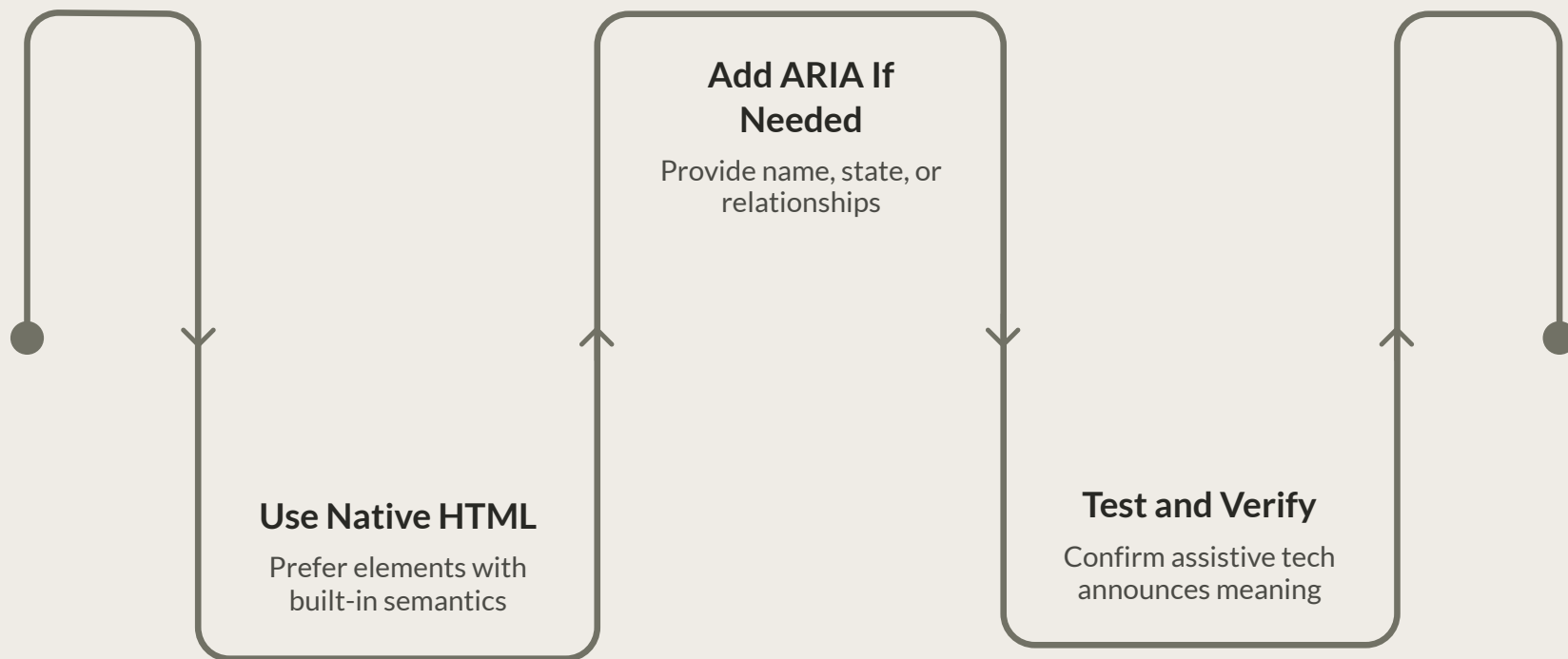
Discussion Prompts

- Which principle had the most violations?
- Which problem would have the greatest user impact?
- Which fix is simplest — and which is most complex?
- What would change if you opened `02-good-page.html` instead?

- ✓ Compare `01-bad-page.html` and `02-good-page.html` side by side after completing your audit to see the implementation difference.

Accessibility Starts with HTML

The single most impactful accessibility decision a frontend developer makes is element choice. Native HTML elements carry built-in accessibility behaviour — keyboard interaction, roles, and states — that no amount of ARIA can fully replicate.



ARIA is a powerful tool, but it is a repair mechanism — not a foundation. Starting with correct HTML means you inherit decades of browser and assistive technology compatibility without writing a single extra attribute.

Semantic Page Structure

A semantically structured page gives screen reader users a navigable architecture — equivalent to a sighted user's ability to scan visually. Without it, every page is a flat wall of text.



Landmark Elements

Use `<header>`, `<nav>`, `<main>`, `<section>`, and `<footer>` to define major page regions. NVDA and JAWS expose these as landmarks users can jump between directly.



Heading Outline

Headings create a navigable document outline. Use heading levels to reflect document hierarchy — not font size or visual weight. A single `<h1>` per page; subheadings nest logically beneath it.



Lists for Grouped Items

Use `<ul>` or `<ol>` for navigation items, related links, and grouped content. Screen readers announce list length, helping users gauge scope before navigating.

Headings Are Navigation

For screen reader users, headings are the primary means of navigating long pages — equivalent to a sighted user scanning visually. Treating headings as styling tools breaks this navigation model entirely.

What Headings Must Do

- Reflect document structure, not visual design preference
- Follow a logical hierarchy: `h1` → `h2` → `h3`
- Never skip levels (e.g. `h1` directly to `h4`)
- Describe the content of the section that follows

What Headings Must Not Do

- Be chosen for font size — use CSS to style any heading level
- Be replaced by bold or large `<p>` text that looks like a heading
- Repeat identical text across multiple sections
- Appear without meaningful content beneath them

 NVDA shortcut: **H** navigates to the next heading. **1-6** navigates by heading level. Use **Insert+F7** to view the full heading outline.

Landmarks Give the Page a Map

HTML landmark elements create a navigation layer that NVDA and JAWS expose directly. Users can jump between landmarks without reading all intervening content — this is the page's equivalent of a visual layout scan.



<header>

Exposes as **banner** landmark. Contains site identity and top-level navigation. One per page.



<nav>

Exposes as **navigation** landmark. Use for primary and secondary navigation blocks. Label multiple navs with **aria-label**.



<main>

Exposes as **main** landmark. Contains the primary page content. Only one **<main>** per page — skip links target it.



<footer>

Exposes as **contentinfo** landmark. Use for site-wide footer content: legal, contact, secondary links.

- 📝 Speaker notes: Ask the participant to navigate 03-structure-exercise-starter.html using NVDA's landmark navigation (D key for next landmark, Shift+D for previous). Ask: "Which landmarks are present? Which are missing?" Typical failures: no main landmark, generic div used instead of nav, multiple unlabelled nav elements. Exercise: add correct landmark elements to the starter file. File: 03-structure-exercise-starter.html → 03-structure-exercise-solution.html for comparison.

Skip Links

Skip links are a simple, high-impact pattern that allows keyboard and screen reader users to bypass repeated navigation blocks and reach the main content immediately. They are among the most cost-effective accessibility improvements a frontend developer can implement.

What a Skip Link Does

- Appears as the first focusable element on the page
- Links directly to the id of the `<main>` element
- Allows users to skip repeated header and navigation content
- Becomes visible on focus — it may be visually hidden until focused

Implementation Requirements

- Target element must have a matching id attribute
- Target `<main>` must be focusable — add `tabindex="-1"` if needed
- Skip link must be keyboard-reachable before other interactive elements
- Do not hide with `display:none` — use a visually-hidden class that shows on focus

Meaningful Links

Link text is read by screen reader users in isolation — through link lists, quick navigation, and tab key browsing. Text that makes sense only in visual context fails these users completely.

Before

Vague links like
"click here" that lack
context



After

Descriptive links
that convey purpose
when read alone

Rule: Link Text Must Make Sense Out of Context

If a user hears only the link text — with nothing before or after it — they must be able to understand where the link goes or what it does.

Rule: Use Links When the User Goes Somewhere

A link navigates to a URL, an anchor on the page, or a new document. If pressing the control triggers an action rather than navigation, use a `<button>`.

Native Controls

The most reliable way to build accessible interactive components is to use the HTML element that was designed for the job. Native elements provide keyboard interaction, focus management, accessible roles, and browser/AT compatibility without additional code.

Use `<button>` When Something Happens

Submitting a form, toggling a panel, opening a modal, triggering an action — these are button behaviours. A `<button>` is keyboard focusable, activated by Enter and Space, and announces as "button" to screen readers automatically.

Use `<a href>` When the User Goes Somewhere

Navigation to a URL or page anchor is a link behaviour. A `<a>` is activated by Enter only, announces as "link", and is listed separately in NVDA/JAWS navigation modes.

Avoid Clickable `<div>` and `<span>`

These elements have no implicit role, no keyboard interaction, and no accessible name. Every clickable div requires manual addition of role, tabindex, keyboard handlers, and ARIA — recreating what a native element provides for free.

Focus Visibility

Keyboard users navigate entirely by focus. If focus is invisible — or jumps unexpectedly — the interface is effectively unusable for a significant portion of users, including those who cannot use a mouse.

→ Keyboard Users Must Know Where They Are

A visible focus indicator is not optional. WCAG 2.2 Success Criterion 2.4.11 (AA) requires focus indicators to have minimum area and contrast. Do not rely solely on browser defaults — they are often insufficient.

→ Do Not Remove Focus Outlines Without Replacement

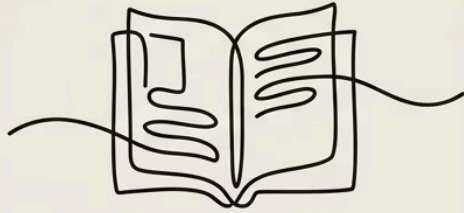
The common CSS rule `:focus { outline: none; }` is one of the most damaging accessibility anti-patterns in frontend code. Always replace it with a clearly visible, high-contrast custom indicator.

→ Focus Order Should Follow Page Logic

The tab sequence must be logical and predictable, matching the visual and reading order. Avoid positive `tabindex` values — they override the natural DOM order and create disorienting focus sequences.

Images and Alt Text

Alt text is not a caption or a description of appearance — it is a functional text alternative that communicates meaning or purpose. The right alt text depends entirely on the image's role in context.



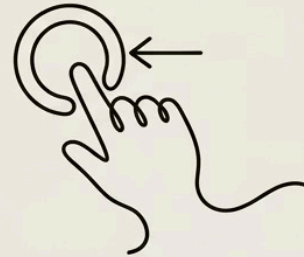
INFORMATIVE IMAGES

**DESCRIBE
MEANING
OR INFO**



DECORATIVE IMAGES

**USE EMPTY
ALT ATTRIBUTE:
alt=""**



FUNCTIONAL IMAGES

**DESCRIBE THE
FUNCTION OR
DESTINATION**

 The most common alt text failure is not missing alt attributes — it is alt text that describes appearance rather than meaning, or alt text that repeats surrounding text verbatim.

ARIA-Lite

ARIA (Accessible Rich Internet Applications) extends HTML's accessibility vocabulary. Used correctly, it communicates name, state, and relationships that native HTML cannot express. Used incorrectly, it creates confusion, contradictions, and silent failures.

What ARIA Can Do

- Communicate accessible name when native labelling is insufficient
- Expose component state: `aria-expanded`, `aria-checked`, `aria-selected`
- Define relationships: `aria-describedby`, `aria-controls`, `aria-owns`
- Assign roles to custom components that require them

What ARIA Cannot Do

- Create keyboard behaviour — ARIA roles do not add key handling
- Fix broken HTML — ARIA on incorrect elements produces unreliable results
- Replace native semantics — overriding a `<button>`'s role with ARIA loses built-in behaviour
- Guarantee identical behaviour across all AT combinations



The first rule of ARIA: do not use ARIA if a native HTML element or attribute provides the required semantics and behaviour.